

Assignment On

‘ADVANCED DATA STRUCTURES AND ALGORITHMS’

(Assignment-4)

Submitted by:
Madhu boini
2503B09904(MCA).

Submitted to:
Dr. Rahul Mishra.

Question:

Simulate Recursive Function Calls Using a Stack. Use Factorial or Fibonacci as an Example.
Display The Function Call Stack.

Code:

```
# Simulating recursive factorial using a manual stack

class Frame:
    def __init__(self, n, stage):
        self.n = n          # parameter for factorial
        self.stage = stage  # 0 = first call, 1 = returning

def simulate_factorial(n):
    stack = []
    result = 1

    # Push the initial call
    stack.append(Frame(n, 0))
    print(f"PUSH: fact({n})")

    while stack:
        frame = stack.pop()

        # Stage 0: first time processing
        if frame.stage == 0:
            if frame.n == 1:
                print("POP : fact(1) = 1")
                result = 1
            else:
                # Push return stage
```

```

        stack.append(Frame(frame.n, 1))

        # Push next recursive call
        stack.append(Frame(frame.n - 1, 0))

        print(f"PUSH: fact({frame.n - 1})")

    # Stage 1: returning from recursion
    else:
        result = frame.n * result
        print(f"POP : fact({frame.n}) = {result}")

    return result

# MAIN PROGRAM
n = 4
print(f"Simulating factorial({n}) using manual stack:\n")
answer = simulate_factorial(n)

print(f"\nFinal Answer = {answer}")

```

Output:

```

PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

Simulating factorial(4) using manual stack:

PUSH: fact(4)
PUSH: fact(3)
PUSH: fact(2)
PUSH: fact(1)
POP : fact(1) = 1
POP : fact(2) = 2
POP : fact(3) = 6
POP : fact(4) = 24

Final Answer = 24

```